Accomplish → SRIM (DigiBull) Web Migration

Technical Report: Conversion, Analysis, and Implementation

DigiBull.ai — SRIM 3.2 Programme

June 2026

Abstract

This document records the migration of the *Accomplish* desktop application (Electron) to a browser-served *local web* application, rebranded as **SRIM** for DigiBull.ai. It describes the original architecture, the initial browser conversion, the subsequent gap analysis and reverse-engineering of a newer packaged release, and the features and fixes implemented to bring the web build to a working state. It also documents the Model Context Protocol (MCP) integration layer, the DigiBull UI update, task-aware Auto model routing, local model download/runtime UX, and how to use the system. The account is factual and neutral; named contributions are recorded only to attribute work.

Contents

1 Original Application (Electron)

The upstream Accomplish app is a pnpm monorepo:

- `apps/desktop` — Electron shell: main process + preload bridge.
- `apps/web` — standalone React UI (Vite + Zustand).
- `apps/daemon` — background Node.js process (no Electron imports).
- `packages/agent-core` — core logic, storage, MCP tooling, the OpenCode agent adapter.

Data flow. The React UI calls `window.accomplish.*` (a bridge of roughly 462 methods). In Electron this was served by the preload `contextBridge` over `ipcRenderer`; the main process forwarded work to the forked daemon, which runs the OpenCode agent runtime and persists state in SQLite (`better-sqlite3`). Native/desktop dependencies include OpenCode, Playwright, Baileys (WhatsApp), `electron-store`, and `electron-updater`.

The renderer (`apps/web/src/client`) contains no Electron or Node-only imports; the desktop coupling lived entirely in the main process, preload, and daemon. This is what made a browser migration feasible.

2 Baseline Artifact and Attribution

The baseline for the converted web project is the archive:

```
LOCAL/SRIM-UI/accomplish_Transformed_Chaithanya_Original.zip
```

This archive is the original Chaithanya transform artifact (approximately 759 MB) and is treated as the starting point for the current SRIM web working tree. Subsequent analysis, gap remediation, UI changes, provider fixes, and documentation updates continue from that baseline. This distinction is important for fair attribution: the web conversion baseline is credited to Chaithanya; the later SRIM verification, remediation, reverse-engineering notes, and current DigiBull UI/model updates are credited separately.

3 Phase 1 — Conversion to a Local Web App

Initial browser conversion (Chaithanya). The Electron window/host was removed so the UI can run in an ordinary browser tab against a local backend. Key additions:

- `apps/daemon/src/browser-api-server.ts` — an HTTP + Server-Sent Events (SSE) server on `127.0.0.1:9234`, letting a browser tab reach the daemon directly without Electron.
- `apps/web/src/client/lib/accomplish-backend.ts` — a browser transport that recreates `window.accomplish` as a `Proxy` over HTTP (`/rpc`) and SSE (`/events`).

- `session.ts` + `LoginPage.tsx` — a login screen using a NetBird Personal Access Token; DigiBull/SRIM branding.
- Daemon dev scripts (tsup build + run) and supporting store changes.
- Delivery baseline: `LOCAL/SRIM-UI/accomplish_Transformed_Chaithanya_Original.zip`.

At the end of this phase the UI loaded in the browser, but only part of the `window.accomplish` surface was wired through the new transport; the remainder defaulted to safe no-ops or stubs.

4 Phase 2 — Verification, Gap Analysis, Reverse-Engineering

Analysis and remediation (Nagabhushana). The converted build was compared against the original repository to establish exactly what was and was not wired.

4.1 Inventory and gap analysis

Two analyses were produced (see Artifacts):

- **migration_inventory.md** — maps every Electron/Node API surface and where it lived.
- **missing_features_gap_analysis.md** — audits the `window.accomplish` contract (∼148 capabilities) against what the browser path actually wires. Result: ∼96 wired, ∼52 missing, bucketed by remediation type:

  A — Trivial-wire (24) backed by an existing daemon service; add a channel.

  B — Needs daemon port (15) logic lived in Electron main.

  C — Browser-native (6) file pickers, clipboard, downloads, capture.

  D — Not possible in browser (7) tray, updater, native window control.

4.2 Reverse-engineering the 0.4.14 release

A packaged installer (`Accomplish-0.4.14-win-x64.exe`, ∼294 MB) was examined to see whether a newer build carried features absent from the source snapshot. Method:

1. Extract the NSIS installer with 7-Zip → `$PLUGINSDIR/app-64.7z`.
2. Extract that archive → `resources/app.asar` (35 MB, 4313 files).
3. Parse the `asar` header (a dependency-free Node script) and diff the IPC channel string literals in the packaged main bundles against the source tree.

Finding. Although the installer's internal version is also `0.1.0`, its code is newer than the source snapshot: ∼124 channel literals appear in the `.exe` but not in source. Verified feature clusters absent from source include an embedded *Browser Panel*, *Triggered Tasks*, local *llama.cpp* inference, *feature flags*, and auto model-routing. Conclusion: these can only be ported from 0.4.14 *source* (the installer contains only minified bundles), and several are native/desktop features. Artifacts of this analysis are in `LOCAL/exe-inspect-0.4.14/` and `exe_0.4.14_feature_delta.md`.

5 Phase 3 — Implemented Features and Fixes

The following were implemented on the converted build. Each was verified at the TypeScript transpile level; full runtime verification requires running the app under Node 24 (see §9).

5.1 Defect fixes

Area	Fix
Agent task execution	The daemon never passed provider API keys to the OpenCode process, so native-provider tasks (OpenAI/Anthropic/...) failed with no credentials. The spawn environment now injects `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, etc. via `buildOpenCodeEnvironment`.
Provider connect crash	`validateApiKeyForProvider` was unwired and returned `null`, so the UI crashed reading `.valid`. Wired to agent-core validation.
Key validation false-negatives	A retired probe model (e.g. `claude-3-haiku`) caused valid keys to be rejected. Validation now treats only `401/403` as invalid (auth), not other statuses.
API-key quoting	Keys pasted with surrounding quotes broke auth headers; a sanitizer strips them on save.
Model listing	`fetchProviderModels` was forwarded with the wrong signature; re-implemented to resolve endpoint + stored key like the original.
Hardcoded addresses	The browser-backend address was hardcoded in 5 places; centralized into one registry (see below).
Silent failures	Failed tasks stored no reason; the failure is now persisted as a system message and visible in history.
Auto routing stalls	Auto model routing is guarded by a timeout. If provider settings or model switching hangs, task submission continues on the current selection and a warning is logged, preventing the UI from staying on *Executing...*.
Provider status nulls	HuggingFace/local provider status is normalized before rendering so a missing server-status response cannot crash the Providers tab.

5.2 New capabilities

Feature	Description
Centralized address registry	`config/endpoints.ts` (client) + `BROWSER_API_PORT`/`WEB_DEV_ORIGIN` (agent-core), env-overridable; production defaults to same-origin — "migration-proof" addressing.
Dev test-login	A local login bypass (token or user/password) for testing without the NetBird service; auto-disabled when `NODE_ENV=production`.

Feature	Description
Bucket-A wiring (24)	Settings config getters/setters, model fetchers, validators, and secrets helpers wired through the browser backend.
Provider parity (bucket B)	Vertex credential save/get, Speech (ElevenLabs) config/validate/transcribe, and OpenAI "Sign in with ChatGPT" OAuth.
Appearance / Customize	Accent palettes, dim (medium) mode, animation toggle, background patterns; persisted client-side.
LOCAL LLM advisor	Detects CPU/RAM (daemon `os`) + GPU/network (browser) and recommends local model sizes; shown atop the providers panel.
DigiBull chat rebrand	Home prompt, placeholder, helper copy, and hover/motion treatment were updated around the DigiBull identity without hardcoding all copy in component logic.
Task-aware Auto model selector	The model selector exposes **Auto · by task type**. At task-submit time the client classifies the prompt (chat, coding, research, planning, analysis), prefers fast/cheap models for simple chat, and stronger models for code, research, and multi-step work. If routing cannot resolve, the existing selected model is used.
Inline AI question card	Question-type permission requests render as an inline **DigiBull Question** card so the agent can ask for clarification and the user can answer without a full-screen modal. File/tool permissions still use the stronger modal.
Local model downloads	A local model download panel sits under the device checker in Providers. It shows selected model state (not selected, ready to download, downloading, installed, failed), download progress, and local server running/idle state.
Runtime target selector	The HuggingFace Local flow exposes Auto, GPU, WebGPU, and CPU runtime targets with helper text. Manual GPU index routing (GPU 0/GPU 1) is not exposed yet; GPU modes use automatic GPU selection.
Performance toggle	`ACCOMPLISH_BROWSER_MODE=none` skips the per-task browser MCP/Chromium spin-up for faster chat startup.
Test-automation skill	A `/test-automation` skill: spec → test cases → run → compare → report.
Provenance	`system:about` channel + visible login credit + `AUTHORS.md` (open, queryable authorship).
Developer mode + MCP manager	A gated Developer section with an MCP connection manager (pre-seeded with the SRIM stack) and an opt-in integration API key.

6 Implementation Coverage

Group	Status	Notes
Bucket A (24)	Done	wired + transpile-verified

Group	Status	Notes
Bucket B portable	Partly done	Vertex creds, Speech, OpenAI OAuth, validation, local model UI/runtime selection
Bucket B native	Open	manual GPU index selection, gcloud Vertex project APIs, remaining native desktop-only services
OAuth (Copilot/Slack)	Open	need provider accounts to build/verify
Bucket C	Partly done	pickers/openExternal native; `exportLogs`, capture* open
Bucket D (7)	N/A	not possible in a browser tab (correctly stubbed)
0.4.14 features	Open	require 0.4.14 source (browser-panel, llama.cpp, etc.)

7 MCP Integration

There are two directions. See `MCP_GUIDE.md` for step-by-step usage.

Outbound (Accomplish → MCP servers). In *Settings* → *General* → *Developer*, enable Developer mode and add remote MCP servers (name + URL). These are stored as connectors and injected into the OpenCode config (`buildMcpServers`), so the agent uses them as tools. The manager is pre-seeded with quick-add templates for the SRIM stack (Dify, FastMCP, MCP Jungle, Postgres, Apache Superset, and the tool MCPs).

Inbound (external workflows → Accomplish). A generated integration API key lets external systems (e.g. Dify/FastMCP) call the local API:

```
POST http://127.0.0.1:9234/rpc
Authorization: Bearer <integration-api-key>
Content-Type: application/json
{ "channel": "task:start", "args": [ { "prompt": "...", "files": [] } ] }
```

The key is honored *only* when the daemon is started with `ACCOMPLISH_ENABLE_INTEGRATION_API=1` (opt-in), and the server binds to `127.0.0.1` only.

8 DigiBull UI and Model UX Update

This update focuses on making SRIM feel understandable while it works, especially when the selected model is automatic or the agent needs one more answer from the user.

8.1 Chat entry and motion

The home page prompt is now DigiBull-branded and the task input bar has a small hover/focus lift, gradient edge, and status pill when Auto mode is active. The motion is intentionally restrained: it is used to show focus and readiness, not to animate every surface.

8.2 Auto model routing

The selector includes **Auto · by task type**. This is not only a label: the client resolves an actual provider/model before `task:start` by reading connected provider settings and classifying

the task prompt. Current heuristics:

- Simple chat prompts prefer cheap/fast models such as Haiku, mini, flash, nano, lite, small, or similar tiers.
- Coding, debugging, implementation, research, scraping, planning, review, and analysis prompts prefer stronger models such as Sonnet, Opus, GPT-4/4o, large/pro/max, 70B+, or similar tiers.
- If the exact desired tier is unavailable, routing falls back through mid-tier or any available connected model.
- If provider settings or model switching hangs, the auto route is skipped after a short timeout and task submission continues.

8.3 Local model download and runtime UX

The Providers panel now places a local model download box directly below the device checker. It lets the user pick a suggested/cached model, download it, run a cached model, stop the local server, and choose a runtime target: **Auto**, **GPU**, **WebGPU**, or **CPU**. The UI shows downloading progress, installed state, failed state, and server running/idle state. Manual GPU index selection (GPU 0/GPU 1) is not supported by the current runtime surface yet; GPU modes use automatic GPU selection.

8.4 AI questions

When the agent sends a question permission request, the execution UI renders a compact inline **DigiBull Question** card with the prompt, optional choices, and a typed-answer field. This makes SRIM's clarification requests visible in the conversation flow instead of looking like a silent hang.

9 Build and Run

```
# 1. Dependencies (Node 24 recommended)
pnpm install
# 2. Backend daemon (optionally enable inbound integration API)
$env:ACCOMPLISH_ENABLE_INTEGRATION_API='1' # optional
pnpm -F "@accomplish/daemon" dev # -> Listening on port 9234
# 3. Web UI
pnpm dev:web # -> http://localhost:5173
```

A convenience launcher (`LAUNCH-WEB.bat`) and a manual checklist (`VERIFY_RESULTS.txt`) are included at the project root.

10 Artifacts

- `LOCAL/SRIM-UI/accomplish_Transformed_Chaithanya_Original.zip` — original Chaithanya web transform baseline archive.
- `LOCAL/migration_inventory.md` — Electron/Node API inventory.

- `.../accomplish_Transformed_Nr/missing_features_gap_analysis.md` — capability gap analysis (buckets A–D).
- `LOCAL/exe_0.4.14_feature_delta.md` — 0.4.14 vs source delta.
- `LOCAL/exe-inspect-0.4.14/` — extracted asar, file list, bundles.
- `LOCAL/docs/MCP_GUIDE.md` — MCP usage guide.
- `LOCAL/docs/DIGIBULL_UI_UPDATE.md` — DigiBull UI, Auto model, local model, and question-card usage notes.